- ***Name: Abdul Alim***
- ***ID :2011176118***
- ***Assignment_01***

problem_1: Write a program in Python to implement blockchain.

```
import hashlib

class  Block:
    def __init__(self,data,previous_hash):
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()
    def calculate_hash(self):
            hash_string = str(self.data)+str(self.previous_hash)+str(self.nonce)
            return hashlib.sha256(hash_string.encode()).hexdigest()
    def nine_block(self,difficulty):
            while self.hash[0:difficulty] !='0' *  difficulty:
                self.nonce += 1
                self.hash = self.calculate_hash()
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.difficulty = 2
    def create_genesis_block(self):
        return Block("Genesis Block" ,"0")
    def get_last_block(self):
        return self.chain[-1]
    def add_block(self,new_block):
        new_block.previous_hash = self.get_last_block().hash
        new_block.nine_block(self.difficulty)
        self.chain.append(new_block)

blockchain = Blockchain()
blockchain.add_block(Block("Block 1", " "))
blockchain.add_block(Block("Block 2", " "))
blockchain.add_block(Block("Block 3", " "))
blockchain.add_block(Block("Block 4", " "))

for block in blockchain.chain:
    print("Block data : ", block.data)
    print("Block hash : ", block.hash)
```

```
Block data :  Genesis Block
Block hash :  8d59df74150242faafaaba4d1632c6e31e4e470c175c2c90c168ec6a4500fc3e
Block data :  Block 1
Block hash :  000ed2983b9272d8536e07beba4678ffba9ac7b3a62d55e31e1843da25144efa
Block data :  Block 2
Block hash :  00172655ca0210f6bf693a27363c99a0c3108ad9257c0fc3fc14dd01e481d34e
Block data :  Block 3
Block hash :  007eb6b94b1874d4bfbeef92412df242bb835183d9860c80750791e99ac3be60
Block data :  Block 4
Block hash :  0059a5d914d50354d4441dd34c95eeaa582c44a036e3b32467d5214d0e0222fe
```

problem_3:Write a program in Python to create four new blocks in a blockchain. Traverse the blocks and print the values.

```
import hashlib
import datetime

class  Block:
    def __init__(self,data,previous_hash):
        self.timestamp = datetime.datetime.now()
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()
    def calculate_hash(self):
            data_string = str(self.timestamp)+str(self.previous_hash)+str(self.data)
            return hashlib.sha256(data_string.encode('utf-8')).hexdigest()

class Blockchain:
    def __init__(self):
```

```
            self.chain = [self.create_genesis_block()]
        def create_genesis_block(self):
            return Block("Genesis Block" ,"0")
        def add_block(self,new_block):
            new_block.previous_hash = self.chain[-1].hash
            self.chain.append(new_block)
        def traverse_chain(self):
             for block in self.chain:
                    print("Timestamp :", block.timestamp)
                    print("Data :", block.data)
                    print("Previous Hash :" , block.previous_hash)
                    print("Hash :" , block.hash)
                    print(" ")

    my_blockchain = Blockchain()
    my_blockchain.add_block(Block("Transection 1", " "))
    my_blockchain.add_block(Block("Transection 2", " "))
    my_blockchain.add_block(Block("Transection 3", " "))
    my_blockchain.add_block(Block("Transection 4", " "))

    my_blockchain.traverse_chain()
```

```
Timestamp : 2025-01-29 05:54:31.188308
Data : Genesis Block
Previous Hash : 0
Hash : ee75aa61819450b6a9abf71a443768ab71ad42445559b2931a31ce5c2ec66982

Timestamp : 2025-01-29 05:54:31.188441
Data : Transection 1
Previous Hash : ee75aa61819450b6a9abf71a443768ab71ad42445559b2931a31ce5c2ec66982
Hash : 1c9a700556fdf549e7b29cf1bcde5dec913bea29f4370cf728e69ee9512d7503

Timestamp : 2025-01-29 05:54:31.188514
Data : Transection 2
Previous Hash : 1c9a700556fdf549e7b29cf1bcde5dec913bea29f4370cf728e69ee9512d7503
Hash : a63e0f2faf69dd395fe59d8fa876da914d07268967c3884d537a866689c28cf3

Timestamp : 2025-01-29 05:54:31.188574
Data : Transection 3
Previous Hash : a63e0f2faf69dd395fe59d8fa876da914d07268967c3884d537a866689c28cf3
Hash : 35a4a512d8e02cf21c30ad1fb070fa7d546500b25775c18413ea8a437570873f

Timestamp : 2025-01-29 05:54:31.188632
Data : Transection 4
Previous Hash : 35a4a512d8e02cf21c30ad1fb070fa7d546500b25775c18413ea8a437570873f
Hash : f652534ebbcbb62f6b670bdf3c14d78530ee8b6932380f68744d272a53d367fd
```

problem_5:Write a program in Python to implement a blockchain and UTXo (unspent transaction output). **bold text**

```
import hashlib
class UTXO:
   def __init__(self, txid, index, value):
     self.txid = txid
     self.index = index
     self.value = value
   def __str__(self):
    return f"UTXO ({self.txid}:{self.index}) with value {self.value}"
class Transaction:
  def __init__(self, inputs, outputs):
    self.inputs = inputs
    self.outputs = outputs
  def __str__(self):
    return f"Transaction with {len(self.inputs)} inputs and {len(self.outputs)} outputs"
  def hash(self):

     tx_input = ''.join([str(inp.txid) + str(inp.index)
     for inp in self.inputs])
     tx_output = ''.join([str(out.value) for out in
     self.outputs])
     tx_data = tx_input + tx_output
     return hashlib.sha256(tx_data.encode('utf-8')).hexdigest()


utxo1 = UTXO('txid1', 0, 10)
utxo2 = UTXO('txid2', 1, 20)
input1 = [utxo1]
```

```
output1 = [UTXO('txid3', 0, 25), UTXO('txid3', 1, 5)]
tx1 = Transaction(input1, output1)
input2 = [utxo2]
output2 = [UTXO('txid4', 0, 15), UTXO('txid4', 1, 5)]
tx2 = Transaction(input2, output2)
print(utxo1)
print(utxo2)
print(tx1)
print(tx2)
print(tx1.hash())
print(tx2.hash())
```

```
UTXO (txid1:0) with value 10
UTXO (txid2:1) with value 20
Transaction with 1 inputs and 2 outputs
Transaction with 1 inputs and 2 outputs
fe3230fe9ee48b845306422ff3c8ae3423fe87ea0cffde0f898813ee6adddd4e
d0b5cddfc541febacd6188b4f15947cc3234e2f590bc3f7cd5a619aa644aa070
```

Problem_7:Write a program in Python to implement PoS consensus algorithm.

```
import hashlib
import time

class Block:
 def __init__(self, data, previous_hash):
  self.timestamp = time.time()
  self.data = data
  self.previous_hash = previous_hash
  self.hash = self.generate_hash()

 def generate_hash(self):
  block_contents = str(self.timestamp) + str(self.data) + str(self.previous_hash)
  block_hash = hashlib.sha256(block_contents.encode()).hexdigest()
  return block_hash
class Blockchain:
 def __init__(self):
  self.chain = [self.create_genesis_block()]

 def create_genesis_block(self):
  return Block("Genesis Block", "0")

 def get_latest_block(self):
  return self.chain[-1]

 def add_block(self, new_block):
  new_block.previous_hash = self.get_latest_block().hash
  new_block.hash = new_block.generate_hash()
  self.chain.append(new_block)

 def is_chain_valid(self):
  for i in range(1, len(self.chain)):
      current_block = self.chain[i]
      previous_block = self.chain[i - 1]
      if current_block.hash != current_block.generate_hash():
        return False
      if current_block.previous_hash != previous_block.hash:
        return False
  return True

if __name__ == '__main__':
 blockchain = Blockchain()

 print("Mining block 1...")
 block1 = Block("Transaction 1", "")
 blockchain.add_block(block1)

 print("Mining block 2...")
 block2 = Block("Transaction 2", "")
 blockchain.add_block(block2)
 print("Mining block 3...")
 block3 = Block("Transaction 3", "")
 blockchain.add_block(block3)
 print("Is blockchain valid? {}".format(blockchain.is_chain_valid()))
```

```
Mining block 1...
Mining block 2...
Mining block 3...
Is blockchain valid? True
```

Problem_9: Write a Python Program that Takes a String and the Desired Number of Leading Zeros from the User and Outputs the Input String, the Nonce Value for Which the Leading Zeros Puzzle Is Solved, and the Corresponding Hash Generated

```
import hashlib

def solvePuzzle(string, leading_zeros) :
    nonce = 0
    while True :
        nonce_str = str(nonce)
        data = string + nonce_str
        hash_value = hashlib.sha256(data.encode()).hexdigest()
        if hash_value.startswith("0" * leading_zeros) :
            return nonce_str, hash_value
        nonce += 1

input_string = input("Enter the string: ")
input_zeros = int(input("Enter the number of leading zeros: "))

nonce_value, hash_result = solvePuzzle(input_string, input_zeros)

print("Input String:", input_string)
print("Nonce value for which the puzzle is solved:", nonce_value)
print("Generated Hash:", hash_result)
```

```
Enter the string: hi
Enter the number of leading zeros: 1
Input String: hi
Nonce value for which the puzzle is solved: 2
Generated Hash: 0251f1ec2880f67631b8d0b3a62cf71a17dfa31858a323e7fc38068fcfaeded0
```

Problem_11: Write a Program in Python to Verify Hash Properties **bold text**

```
import hashlib

message = b"Hello, world!"

md5_hash = hashlib.md5(message).hexdigest()
sha1_hash = hashlib.sha1(message).hexdigest()
sha256_hash = hashlib.sha256(message).hexdigest()

if md5_hash == hashlib.md5(message).hexdigest() :
    print("MD5 hash is consistent")
else :
    print("MD5 hash is inconsistent")



if len(md5_hash) == 32 :
    print("MD5 hash is 32 characters long")
else :
    print("MD5 hash is not 32 characters long")


if sha1_hash == hashlib.sha1(message).hexdigest():
    print("SHA-1 hash is consistent")
else:
    print("SHA-1 hash is inconsistent")


if len(sha1_hash) == 40:
    print("SHA-1 hash is 40 characters long")
else:
    print("SHA-1 hash is not 40 characters long")


if sha256_hash == hashlib.sha256(message).hexdigest():
    print("SHA-256 hash is consistent")
else:
    print("SHA-256 hash is inconsistent")
```

```
if len(sha256_hash) == 64:
    print("SHA-256 hash is 64 characters long")
else:
    print("SHA-256 hash is not 64 characters long")
```

```
MD5 hash is consistent
MD5 hash is 32 characters long
SHA-1 hash is consistent
SHA-1 hash is 40 characters long
SHA-256 hash is consistent
SHA-256 hash is 64 characters long
```

Problem_13: Write a Python program to Demonstrate the Mining Process in Blockchain

```
import hashlib
import time

version = 1
previous_block_hash = "000000000000000000007d28e1a9ac3b37760e3b3fbbd3df2b8f7670a434f18a6"
merkle_root = "9d7d1c2fa42e7f520d33de8e7bb28132586d30ef7c6d9b9e446e6c12d1f7cf25"
timestamp = int(time.time())
difficulty = 4
nonce = 0
header = str(version) + previous_block_hash + merkle_root + str(timestamp) + str(difficulty) + str(nonce)
while True :
    header_with_nonce = header + str(nonce)
    hash = hashlib.sha256(header_with_nonce.encode()).hexdigest()
    if hash[:difficulty] == "0" * difficulty :
        print("Block mined successfully!")
        print("Nonce:", nonce)
        print("Hash:", hash)
        break
    nonce += 1
```

```
Block mined successfully!
Nonce: 91740
Hash: 000077c845872679d4778117c19af06b25cf4a323a2c309e0c47e01f8496e854
```